Flock Energy — Engineering Take-Home Submission

Urja Meter Ops Clean API Gateway & Reverse-Engineering Protocol

PUBLIC GITHUB REPOSITORY LINK

https://github.com/Atom0204/Prj-flock

Clone command: `git clone https://github.com/Atom0204/Prj-flock.git`

Candidate: Atom0204 (shivamkashyap0204@gmail.com)

Role: Backend / Full-Stack Engineer Take-Home

Target System: Urja Meter Ops (https://urja-ops.flockenergy.tech)

Stack: Python 3.12, FastAPI, Pydantic v2, Pytest, Leaflet, Chart.js

Submission Table of Contents

- **Part 1: Front-Door Guide (README)** — Architecture, installation, how to run, sample requests, trade-offs, and scaling analysis.
- **Part 2: Reverse-Engineering Protocol (PROTOCOL.md)** — Detailed analysis of portal auth, SvelteKit CSRF headers, 1-hour session tokens, HMAC-SHA256 export signing, and legacy vs v2 schema normalization.
- **Part 3: Engineering Reflection (REFLECTION.md)** — Complete answers to all evaluation reflection prompts (assumptions, obstacles, what would be improved with another day, mistakes made, self-critique).
- **Part 4: Optional Extensions Delivered** — Full implementation of all 6 optional extensions (Network hierarchy tree, local query/search layer, geospatial Haversine proximity, session auto-recovery, HMAC bulk fast-path, and modern dashboard UI).

Part 1: Project README & System Guide

Urja Meter Ops — Clean API Gateway Service



A production-ready, asynchronous RESTful API gateway built in front of the legacy **Urja Meter Ops** internal utility portal (`https://urja-ops.flockenergy.tech`).

This service reverse-engineers the portal's SvelteKit session protocol, CSRF defenses, and cryptographic HMAC export mechanisms to provide a clean, documented, strongly-typed JSON API for product and data engineering teams.

⚡ Quick Navigation

- 📖 **Reverse-Engineering Protocol Write-Up**: PROTOCOL.md *(How the portal actually works under the hood)*
- 🧠 **Engineering Reflection & Self-Critique**: REFLECTION.md *(Assumptions, challenges, mistakes & lessons)*
- 📄 **OpenAPI 3.1 Specification**: openapi.json *(Full machine-readable API contract)*
- 🌐 **Interactive Documentation**:
- **Swagger UI**: http://localhost:8000/docs
- **Scalar Reference**: http://localhost:8000/scalar
- **ReDoc**: http://localhost:8000/redoc
- 📊 **Modern Web Client Dashboard**: http://localhost:8000/ui *(Interactive fleet analytics, charts, hierarchy tree & map)*

🎯 What Was Built



Core Features:

1. **Full Smart Meter Directory**: Fast, paginated listing with multi-attribute filtering across Manufacturer (`make`), Phase (`single` / `three`), Status (`Installed` / `Faulty` / `Decommissioned`), Connection Type, and DT Code.
2. **Reconstructed 7-Tier Network Hierarchy**: Automatically aggregates meters into the utility's distribution tree: `Zone` → `Circle` → `Division` → `Subdivision` → `Substation` → `Feeder` → `Distribution Transformer (DT)` .

3. **Consumption & Voltage Analytics**: Granular 30-minute interval readings (kWh, kVAh, Voltage) with calculated net energy consumption deltas, voltage minimum/maximum/averages, and ISO 8601 timestamps.
4. **Geospatial Proximity Search**: Instant geodesic (Haversine) distance queries finding meters within X km of any given latitude/longitude.
5. **Schema Reconciliation**: Transparently normalizes divergent `legacy` parameter arrays and modern `v2` escaped JSON class data into a unified dictionary.
6. **Robust Upstream Session Management**: Automatic authentication on startup, periodic cookie refresh, and transparent 401 retry handling so consumers never experience upstream token drops.
7. **Modern Web Client**: A rich single-page dashboard featuring KPI cards, manufacturer distribution doughnut charts, interactive filterable meter table, meter inspection drawer with Chart.js consumption curves, hierarchy tree view, and Leaflet.js spatial map!

🚀 Quickstart & Installation

1. Prerequisites

- Python 3.10+ (tested on Python 3.11 and 3.12)
- `pip`

2. Clone Repository & Install Dependencies

```
git clone <repository_url>
cd Flock

# Optional: Create and activate virtual environment
python -m venv .venv
# On Windows:
.venv\Scripts\activate
# On Linux/macOS:
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

3. Configuration (`.env`)

The service comes pre-configured with the default Urja credentials. To modify settings, create or edit `.env`:

```
# Portal Credentials
URJA_PORTAL_URL=https://urja-ops.flockenergy.tech
URJA_USERNAME=operator@urja.local
URJA_PASSWORD=urja-ops-2026

# Server Settings
HOST=0.0.0.0
PORT=8000
DEBUG=false

# Cache & Sync Settings
CACHE_TTL_SECONDS=300
AUTO_SYNC_ON_STARTUP=true
SESSION_REFRESH_BUFFER_SECONDS=120
```

4. Run the Service

```
python run.py
```

Alternatively, run with Uvicorn directly:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

📡 Sample Requests & Usage

1. List & Filter Meters

```
# Search by query, filter by make and status
curl -X GET "http://localhost:8000/api/v1/meters?make=Secure&status=Installed&page=1&page_size=5"
```

Sample Response:

```
{
  "items": [
    {
      "meterId": "J100003",
      "serialNo": "SE47035",
      "make": "Secure",
      "phaseType": "single",
      "installStatus": "Installed",
      "installType": "Whole Current",
      "build": "legacy",
      "dtCode": "DT-001",
      "geo": {
        "latitude": 26.93896,
        "longitude": 75.83095
      }
    }
  ],
  "total": 48,
  "page": 1,
  "page_size": 5,
  "total_pages": 10
}
```


2. Detailed Meter Metadata & Normalized Nameplate

```
curl -X GET "http://localhost:8000/api/v1/meters/J100000"
```

Sample Response:


```
{
  "meterId": "J100000",
  "serialNo": "SE33962",
  "make": "HPL",
  "phaseType": "single",
  "installStatus": "Decommissioned",
  "installType": "Whole Current",
  "build": "legacy",
  "dtCode": "DT-001",
  "geo": {
    "latitude": 26.93896,
    "longitude": 75.83095
  },
  "hierarchy": {
    "zone": { "name": "Jaipur Zone 1", "code": "Z-01" },
    "circle": { "name": "Circle 1", "code": "C-01" },
    "division": { "name": "Division 1", "code": "D-01" },
    "subdivision": { "name": "Subdivision 1", "code": "SD-01" },
    "substation": { "name": "Substation 1", "code": "SS-01" },
    "feeder": { "name": "Feeder 1", "code": "F-001" },
    "dt": { "name": "Malviya Nagar DT 1", "code": "DT-001" }
  },
  "nameplate_attributes": {
    "Meter ID": "J100000",
    "Serial No": "SE33962",
    "Make": "HPL",
    "Phase Type": "single",
    "Installation Status": "Decommissioned",
    "Installation Type": "Whole Current"
  },
  "energy_summary": {
    "readings_count": 337,
    "kwh_consumption": 142.05,
    "min_voltage": 220.0,
    "max_voltage": 240.0,
    "avg_voltage": 229.8
  }
}
```


3. Consumption & Voltage Time-Series Readings

```
curl -X GET "http://localhost:8000/api/v1/meters/J100000/energy?limit=3"
```

Sample Response:


```
{
  "meter_id": "J100000",
```

```
  "readings_count": 3,
  "summary": {
    "readings_count": 337,
    "kwh_consumption": 142.05,
    "min_voltage": 220.0,
    "max_voltage": 240.0,
    "avg_voltage": 229.8
  },
  "readings": [
    {
      "timestamp": "30/06/2026 23:00",
      "iso_timestamp": "2026-06-30T23:00:00+00:00",
      "kwh": 48580.36,
      "kvah": 52466.79,
      "volt_r": 230.0
    },
    {
      "timestamp": "30/06/2026 23:30",
      "iso_timestamp": "2026-06-30T23:30:00+00:00",
      "kwh": 48580.79,
      "kvah": 52467.25,
      "volt_r": 220.0
    }
  ]
}
```


4. Geospatial Proximity Search

```
# Find meters within 2.5 km of target coordinates
curl -X GET "http://localhost:8000/api/v1/meters/nearby?lat=26.9389&lng=75.8309&radius_km=2.5&limit=5"
```

5. Distribution Network Hierarchy Tree

```
curl -X GET "http://localhost:8000/api/v1/hierarchy"
```

6. Fleet Overview Statistics

```
curl -X GET "http://localhost:8000/api/v1/stats"
```

🧪 Running Automated Tests

A comprehensive test suite is included covering data models, coordinate math, hierarchy tree generation, API endpoints, and client authentication:

```
# Run tests with pytest
python -m pytest tests/ -v
```

Output:

```
tests/test_api.py::test_list_meters PASSED
tests/test_api.py::test_filter_meters_by_make PASSED
tests/test_api.py::test_get_meter_detail PASSED
tests/test_api.py::test_get_meter_not_found PASSED
tests/test_api.py::test_get_stats PASSED
tests/test_api.py::test_nearby_meters PASSED
tests/test_client.py::test_hmac_signature_generation PASSED
tests/test_client.py::test_urja_client_initialization PASSED
tests/test_hierarchy.py::test_hierarchy_tree_reconstruction PASSED
tests/test_hierarchy.py::test_hierarchy_tree_handles_missing_data PASSED
tests/test_schemas.py::test_meter_summary_aliases PASSED
tests/test_schemas.py::test_haversine_distance PASSED
tests/test_schemas.py::test_timestamp_parsing PASSED
====================== 13 passed in 1.13s ======================
```

📁 Project Structure

```
Flock/
├── app/
│   ├── __init__.py
│   ├── main.py                 # FastAPI app configuration, lifespan, and static routing
```

```
│   ├── config.py               # Pydantic BaseSettings environment configuration
│   ├── client/
│   │   ├── __init__.py
│   │   └── urja_client.py      # Reverse-engineered async portal client with auth & HMAC signing
│   ├── models/
│   │   ├── __init__.py
│   │   └── schemas.py          # Pydantic v2 schemas for meters, readings, and hierarchy
│   ├── services/
│   │   ├── __init__.py
│   │   ├── meter_service.py    # In-memory indexing, multi-attribute querying & proximity
│   │   └── hierarchy.py        # 7-tier distribution hierarchy reconstruction
│   ├── api/
│   │   ├── __init__.py
│   │   └── endpoints.py        # RESTful API route definitions
│   └── static/
│       └── index.html          # Modern Web Client UI (Chart.js + Leaflet.js dashboard)
├── tests/
│   ├── __init__.py
│   ├── test_api.py             # API endpoint integration tests
│   ├── test_client.py          # HMAC signature and client logic unit tests
│   ├── test_hierarchy.py       # Network hierarchy reconstruction tests
│   └── test_schemas.py         # Coordinate math and model parsing tests
├── scripts/
│   └── generate_openapi.py     # Automated generator for openapi.json
├── openapi.json                # Exported OpenAPI 3.1 specification
├── PROTOCOL.md                 # Comprehensive write-up of portal reverse engineering
├── REFLECTION.md               # Detailed engineering reflection & self-critique
├── requirements.txt            # Pinned Python package dependencies
├── run.py                      # Service entrypoint launcher
├── .env.example                # Example configuration template
└── README.md                   # System documentation (you are here)
```

⚖️ Design Decisions & Trade-Offs

Decision	Rationale	Trade-Off
HMAC Bulk Export Ingestion	The portal provides a cryptographically signed `/portal/export` endpoint that returns all 403 meters at once in ~200ms.	Requires reverse-engineering the HMAC-SHA256 signing algorithm rather than scraping HTML or crawling 21 search pages.
In-Memory Query Indexing	For 403 meters, in-memory filtering delivers sub-millisecond query responses and zero external database operational overhead.	In-memory indexing is bounded by RAM. At scale ($>500,000$ meters), this should be migrated to PostgreSQL + PostGIS.
Automatic 401 Session Interceptor	The upstream portal expires sessions after 1 hour (`Max-Age=3600`). Re-authenticating transparently prevents dropped requests.	Introduces a 200ms latency spike once every hour for whichever request triggers the renewal.
Dual Schema Normalization	The portal has two conflicting formats (`data` parameter array for legacy vs stringified JSON in `classData` for v2).	We reconcile both into a uniform `nameplate_attributes` dictionary, shielding consumers from upstream technical debt.

🚫 What Was Intentionally Left Out & What We'd Improve Next

Intentionally Skipped:

- **Direct Database Storage (PostgreSQL)**: With 403 smart meters and 40 transformers, requiring PostgreSQL or Docker would add unnecessary setup friction for reviewers. Python's in-memory data structures provide higher performance without infrastructure dependencies.
- **Write Operations (PUT/POST/DELETE on meters)**: As noted in the ground rules, the upstream portal represents an operational utility system and must be treated as strictly read-only.

Next Steps with More Time:

- **Time-Series Engine (TimescaleDB / ClickHouse)**: For storing millions of 30-minute interval readings over multi-year spans and calculating complex feeder loss balances.
- **Proactive Webhooks & Anomaly Detection**: An alert engine detecting voltage sags ($<210\text{V}$), tamper indicators, or power outages and publishing events to webhooks.
- **PostGIS Native Spatial Indexing**: R-tree bounding box indexing for ultra-high-density urban meter grids.

Part 2: PROTOCOL.md — Reverse Engineering Analysis

Reverse-Engineering Protocol Documentation: Urja Meter Ops Portal

This document outlines the internal architecture, network protocols, authentication mechanisms, data models, and reverse-engineered behavior of the legacy **Urja Meter Ops** portal (`https://urja-ops.flockenergy.tech`).

---

1. System Overview & Technology Stack

The Urja Meter Ops portal is an internal web application used by utility operations teams to inspect smart meter telemetry, verify electrical distribution hierarchy, and review interval energy readings.

Inspection of client bundles and server responses revealed the underlying technology stack: * **Frontend Framework**: **SvelteKit** (compiled with Vite, utilizing modern Svelte 5 runes and client-side page routing under `_app/immutable/`). * **Authentication Provider**: **Better-Auth** (`__Secure-better-auth.session_token`). * **Server-Side Rendering**: SvelteKit SSR delivering HTML with embedded dehydrated state (`node_ids: [0, 2, 5]`) and dehydrated route data accessible via `/__data.json` endpoints serialized with the **Devalue** protocol. * **Internal API Namespace**: `/portal/*` (JSON-backed endpoints).

---

2. Authentication & Session Lifecycle

2.1 The Login Flow

- **Endpoint**: `POST /login`
- **Content-Type**: `application/x-www-form-urlencoded`
- **Form Parameters**:
- `email` (string): The operator's email (e.g., `operator@urja.local`). *Note: The prompt specified "Username", but the actual HTML input name is* `email` .
- `password` (string): Operator password (`urja-ops-2026`).

2.2 SvelteKit CSRF Protection (Key Quirk)

Initial direct `POST` requests via script resulted in HTTP **403 Forbidden** with the body:

```
Cross-site POST form submissions are forbidden
```

Root Cause: SvelteKit strictly validates the `Origin` header against the application host for all form actions to mitigate CSRF attacks. Automated clients that omit the `Origin` header (such as standard curl or unconfigured HTTP client libraries) are immediately rejected. **Resolution**: The client must explicitly send:

```
Origin: https://urja-ops.flockenergy.tech
Referer: https://urja-ops.flockenergy.tech/login
```

2.3 Session Cookie & Token Expiry

Upon valid authentication, the portal responds with HTTP `200 OK` (or `303 See Other`) and JSON:

```
{
  "type": "redirect",
  "status": 303,
  "location": "/meters"
}
```

Along with the `Set-Cookie` header:

```
Set-Cookie: __Secure-better-auth.session_token=<SESSION_TOKEN>; Max-Age=3600; Path=/; HttpOnly; Secure; SameSite=Lax
```

- **Cookie Lifetime**: Exactly 3,600 seconds (1 hour).
- **Unauthorized Behavior**: Unauthenticated requests or expired sessions to `/portal/*` yield: `json HTTP/1.1 401 Unauthorized {"error": "unauthorized", "message": "A valid session is required."}`
- **Gateway Mitigation**: Our `UrjaClient` implements token expiry tracking with a 120-second safety buffer and transparent 401-intercepting automatic re-login with mutex locking.

---

3. Discovered Endpoints & Protocols

Reverse-engineering the compiled JavaScript chunks (`5.C4dYcCRt.js` , `4.2Bgc2kUI.js` , `6.l777LXKL.js`) uncovered the following operational routes:

3.1 Meter Directory Search

- **Endpoint**: `GET /portal/meters/search?q={query}&page={page}`
- **Response**: `json { "data": [ { "meterId": "J100000", "serialNo": "SE33962", "make": "HPL", "phaseType": "single", "installStatus": "Decommissioned", "dtCode": "DT-001" } ], "total": 403, "page": 1, "pageSize": 20 }`
- **Characteristics**:
- Paginates 20 records per page.
- Free-text search on `q` matches against meter ID, hardware serial number, and make.

3.2 Distribution Transformers (DTs)

- **Endpoint**: `GET /portal/dts?page={page}`
- **Response**: `json { "data": [ { "code": "DT-001", "name": "Malviya Nagar DT 1", "feederCode": "F-001", "capacityKva": 100 } ], "total": 40, "page": 1, "pageSize": 20 }`
- **Characteristics**:
- Total of 40 distribution transformers across 2 pages.
- Maps each DT to a parent `feederCode` and rated `capacityKva` .

3.3 Cryptographic Key Distribution

- **Endpoint**: `GET /portal/keys`
- **Response**: `json { "data": { "signingSecret": "I3dZPPf5CgTp7JyGNMI8i6z8LFR7TmSR" } }`
- **Purpose**: Distributes an HMAC-SHA256 shared secret used by the browser to sign bulk export requests.

3.4 Cryptographically Signed Bulk Export (The Fast Path)

- **Endpoint**: `GET /portal/export?page=1`
- **Required Headers**:
- `x-timestamp` : Unix epoch timestamp in seconds (e.g., `1730187900`).
- `x-signature` : Hex-encoded HMAC-SHA256 signature.
- **Signing Algorithm**: `text canonical_string = f"{HTTP_METHOD}\n{PATH}\n{QUERY_STRING}\n{TIMESTAMP}" signature = HMAC_SHA256(key=signingSecret, message=canonical_string).hexdigest()` *Example*: For `GET` , `/portal/export` , `page=1` , timestamp `1730187900` : `text GET /portal/export page=1 1730187900`
- **Payload Structure**: Returns all **403 smart meters in a single HTTP response**, containing comprehensive metadata: `json { "total": 403, "data": [{ "meterId": "J100000", "serialNo": "SE33962", "make": "HPL", "phaseType": "single", "installStatus": "Decommissioned", "installType": "Whole Current", "build": "legacy", "dtCode": "DT-001", "hierarchy": { "zone": { "name": "Jaipur Zone 1", "code": "Z-01" }, "circle": { "name": "Circle 1", "code": "C-01" }, "division": { "name": "Division 1", "code": "D-01" }, "subdivision": { "name": "Subdivision 1", "code": "SD-01" }, "substation": { "name": "Substation 1", "code": "SS-01" }, "feeder": { "name": "Feeder 1", "code": "F-001" }, "dt": { "name": "Malviya Nagar DT 1", "code": "DT-001" } }, "geo": { "lat": 26.938961002479868, "lng": 75.83095696146852 } }] }`

3.5 Meter Geolocation Coordinates

- **Endpoint**: `GET /portal/meters/{meterId}/geo`
- **Response**: `json { "data": { "latitude": "26.938961002479868", "longitude": "75.83095696146852" } }`
- **Error**: Returns `404 Not Found` with `{"error": "not_found", "message": "Meter not found"}` if the meter does not exist.

3.6 Interval Energy Consumption Readings

- **Endpoint**: `GET /portal/meters/{meterId}/energy`
- **Response**: `json { "data": [ { "timestamp": "23/06/2026 23:30", "kwh": "48438.74", "kvah": "52313.84", "voltR": "226" }, ... ] }`
- **Characteristics**:
- Typical meter logs 337 readings covering 7 days of 30-minute intervals ($7 \times 48 = 336 + 1$ boundary reading).
- Timestamps use the human-readable format `DD/MM/YYYY HH:MM` .
- Readings are returned as strings (`"48438.74"` , `"226"`) rather than native floating-point numbers.

---

4. Discovered Quirks, Surprises & Inconsistencies

4.1 Schema Divergence: `legacy` vs `v2` Telemetry Builds

When inspecting individual meter detail pages (`/meters/{id}/__data.json`), we discovered two mutually incompatible data representations generated by different generations of smart meters:

1. `legacy` **Build (238 meters)**:

2. Attributes are stored in an array of parameter objects: `json "detail": { "data": [{ "parameterName": "Meter ID", "parameterValue": "J100000" }, { "parameterName": "Serial No", "parameterValue": "SE33962" }, { "parameterName": "Make", "parameterValue": "HPL" }, { "parameterName": "Phase Type", "parameterValue": "single" }, { "parameterName": "Installation Status", "parameterValue": "Decommissioned" }, { "parameterName": "Installation Type", "parameterValue": "Whole Current" }] }`
3. `v2` **Build (165 meters)**:
4. Attributes are packaged as an escaped JSON string nested under `classData`: `json "detail": { "classData": "{\"installed_meter\": {\"MeterId\":\"J100004\",\"SerialNo\":\"SE65293\",\"Make\":\"Genus\",\"PhaseType\":\"single\",\"InstallationStatus\":\"Faulty\",\"InstallationType\":\ Operated\"}}" }`
5. **How our API service handles this**: Our `MeterService` transparently normalizes both schemas into a consistent `nameplate_attributes` dictionary so API consumers never have to deal with raw format discrepancies.

4.2 Non-Standard Timestamp Formatting

The energy telemetry uses Indian/UK standard date strings (`DD/MM/YYYY HH:MM`) without timezone indicators. Our API parses these into strict ISO 8601 UTC strings (`YYYY-MM-DDTHH:MM:00Z`) while preserving the original timestamp for auditability.

4.3 Meter Reading Cadence Variations

While modern smart meters report 337 readings across the 7-day period (30-minute interval profile), certain legacy meters (such as `J100050`) report only daily cumulative totals (8 readings across 8 days). Our energy summary engine handles varying sampling frequencies without crashing.

4.4 Data String Types

Numeric fields such as `latitude` , `longitude` , `kwh` , `kvah` , and `voltR` are returned by the portal as strings (e.g. `"226"` , `"48438.74"`). Our Pydantic models automatically cast and validate them as typed Python floats.

Part 3: REFLECTION.md — Engineering Reflection & Self-Critique

Engineering Reflection: Urja Meter Ops Gateway

---

1. What assumptions did you make?

1. **Read-Only Upstream & Session Longevity**: We assumed the Urja Meter Ops portal is an internal operational system that does not undergo high-frequency writes during operational hours. Given the 1-hour session cookie (`Max-Age=3600`) and the presence of an HMAC-signed bulk export endpoint, we assumed an in-memory index synchronized periodically (default TTL: 300 seconds) provides an optimal balance between freshness, upstream portal protection, and sub-millisecond query latency.
2. **Geographical Coordinates Reference System**: We assumed the coordinates provided by `/portal/meters/{id}/geo` (`latitude: ~26.93` , `longitude: ~75.83`) are in standard WGS84 decimal degrees (placing the fleet in Jaipur, Rajasthan, India) and that great-circle (Haversine) spherical distance approximations are sufficiently accurate for local utility queries up to 50 km.
3. **Hierarchy Completeness & Granularity**: We assumed the utility follows a strict 7-tier distribution hierarchy: `Zone` → `Circle` → `Division` → `Subdivision` → `Substation` → `Feeder` → `Distribution Transformer (DT)` . Where meters have missing tiers in edge cases, we assumed generating synthetic placeholder nodes (`UNKNOWN_DIVISION`) is preferable to dropping records or raising 500 errors.
4. **Timezone Interpretation**: Since the portal provides timestamps in the unadorned format `DD/MM/YYYY HH:MM` for an Indian power utility, we assumed UTC/IST normalization without offset shifts, preserving the original string while emitting ISO 8601 strings.

---

2. Which part was the most difficult, and how did you get unstuck?

The most challenging obstacle was **uncovering the HMAC-SHA256 signature scheme** required to access the `/portal/export` endpoint.

- **Initial Discovery**: Initially, navigating the portal gave the impression that meters could only be queried through the paginated `/portal/meters/search` endpoint (20 items at a time across 21 pages).
- **Investigation**: By downloading and inspecting the compiled Vite/SvelteKit frontend bundles (specifically `4.2Bgc2kUI.js` and `DHWw2r--.js`), we observed a client-side Web Crypto call `crypto.subtle.sign('HMAC', ...)` tied to an "Export" button on the Transformers page.
- **The Breakthrough**: Deobfuscating the JavaScript revealed that the portal fetches a secret key from `/portal/keys` and signs a newline-delimited canonical string: `text GET /portal/export page=1 <unix_timestamp_seconds>` And attaches `x-timestamp` and `x-signature` headers. Implementing this exact algorithm in Python immediately unlocked the fast-path full dataset retrieval in a single 200ms request!

---

3. If you had another day, what would you improve?

1. **Persistent Relational / Spatial Database (PostgreSQL + PostGIS)**: While our current in-memory data store is lightning fast for 403 meters, a utility managing 500,000+ smart meters would outgrow process memory. We would introduce SQLite/PostgreSQL with PostGIS for native spatial indexing (`ST_DWithin` , R-tree indexes) and SQL-based complex joins across the hierarchy.
2. **Historical Time-Series Database Integration (TimescaleDB / ClickHouse)**: Energy interval readings ($337 \text{ readings} \times 403 \text{ meters} = 135,000+ \text{ data points}$) grow continuously. Moving interval data to a time-series engine would enable analytical rollups (e.g., peak load, aggregate DT feeder losses, transformer overload risk alerts).
3. **Webhook & Anomaly Detection Pipeline**: We would build a proactive background monitor detecting voltage sags ($< 210\text{V}$), phase unbalance, or tamper states (meters switching to `Faulty`), triggering Webhook alerts to utility dispatchers.
4. **OAuth2 / API Key Rate Limiting Layer**: Add client API key management with token bucket rate limiting (e.g., via Redis) to guard the gateway against rogue internal scripts.

---

4. What mistake did you make while solving this?

When first testing form submission to `/login` via Python's `requests` library, the portal returned HTTP `403 Forbidden: Cross-site POST form submissions are forbidden` .

Initially, we assumed the portal required a hidden CSRF token or session cookie to be scraped from the login page HTML. After wasting a few minutes searching the DOM for hidden `<input type="hidden" name="csrf">` fields (which didn't exist), we realized this was SvelteKit's built-in `checkCrossOrigin` server protection. Because standard HTTP libraries don't include an `Origin` header on requests by default, SvelteKit flagged it as an illegal cross-site submission. Adding `"Origin": "https://urja-ops.flockenergy.tech"` immediately resolved the issue.

A secondary bug occurred in our initial pytest run where a `NameError: UrjaClientError` was encountered because `UrjaNotFoundError` was imported without its parent class, resulting in an unhandled 500 during a 404 test. Fixing the import restored all 13 unit tests to green.

---

5. If you were reviewing your own submission, what would you criticise?

1. **Memory Growth at Massive Scale**: Our decision to cache all 403 meters and their hierarchy in process RAM delivers blazing $< 2\text{ms}$ response times, but is fundamentally memory-bound. If the utility fleet scaled to 5 million meters, holding raw dictionaries in RAM would cause substantial memory pressure. We clearly documented this scaling boundary in the README.
2. **On-Demand Scraping for Meter Nameplates**: Because the `/portal/export` endpoint provides most attributes but leaves deep SSR nameplate attributes inside `/meters/{id}/__data.json`, fetching full details for an uncached meter requires an on-demand HTTP fetch to the portal. While we added a nameplate LRU cache, a batch-crawler warming this on startup would ensure zero-latency cold hits.
3. **Coupling to Specific Portal Frontend Routes**: Relying on `/meters/{id}/__data.json` means that if the upstream utility rebuilds their SvelteKit frontend and changes route bundle hashes or SSR serialization, the nameplate parser might need updating. A robust production deployment would use integration tests scheduled against a staging instance to flag any upstream bundle changes.